ICT162

Timed Online Assignment – July Semester 2021

Object Oriented Programming

Thursday, 25 November 2021 **1:00 pm – 3:30 pm**

Time allowed: 2.5 hours

INSTRUCTIONS TO STUDENTS:

1. This Timed Online Assignment (TOA) contains **FOUR (4)** questions and comprises **ELEVEN (11)** pages (including cover page).

2. You must answer **ALL** questions.

3. If you have any queries about a question, or believe there is an error in the question, briefly explain your understanding and assumptions about that question before attempting it.

4. You MUST **submit your answers via Canvas (similar to TMA submission) at the end time of this TOA** (as stated on this cover page). The 15 minutes grace period as shown on Canvas is strictly meant for technical issues encountered during submission. Thereafter, you will not be able to submit your answers and you will be considered as having withdrawn from the course. **No appeal will be allowed.**

5. You are to include the following particulars in your submission: TOA Course Code, Your Name, and SUSS PI No. Name your submission file as TOACourseCode_FullName_StudentPI. Use underscore and NOT Space. *Example: TOAICT162_RaphaelLee_T1923161 (omit D/O, S/O).*

6. Your submission should consist of only one file and **must not exceed 500MB** in size. The file must be a Microsoft Word file saved in .doc or .docx format. All answers are to be typed. Flowcharts and graphs may be scanned or photographed and embedded in the Word file provided it does not exceed the file size limit of 500MB. Images of handwritten answers will not be marked.

7. To prevent plagiarism and collusion, your submission will be reviewed by Turnitin. The Turnitin report will only be made available to the marker and you will not be able to view it.

8. The University takes plagiarism and collusion seriously, and your Turnitin report will be examined thoroughly as part of the marking process.

Answer all questions. (Total 100 marks)

Question 1

Study the class diagram in Figure Q1 below. The Player and Game classes are described in Table Q1(a) and Table Q1(b) respectively.

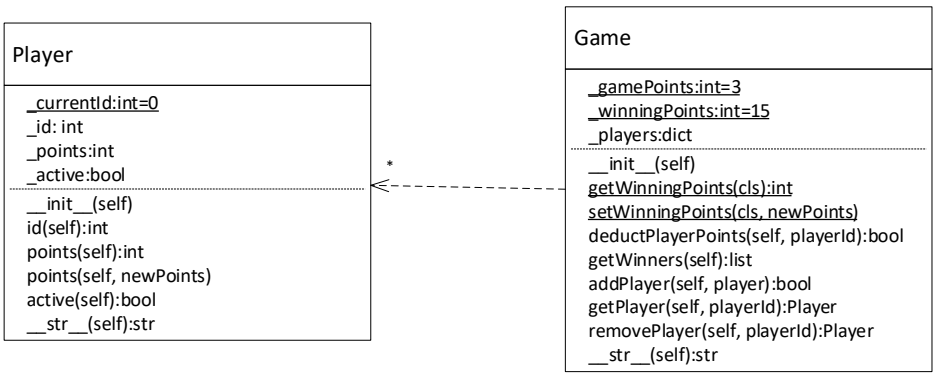

Figure Q1

The Player class has the following attributes: • _currentId is used to auto generate running numbers for the id of players. • _id uniquely identifies a player. • _points records the current points of a player • _active records whether a player is active or otherwise. A player becomes inactive when he has zero point.

The Player class has the following methods:
• __init__(self) initialises a new player with
  – an auto-generated player Id starting from 1,
  – points set to 5
  – active set to true
• getter methods for id, points and active
• a setter method for points. Do not let point be negative. If the parameter newPoints is negative, set points to 0. Whenever points become zero, the player is no longer active.
• __str__(self) returns a string representation of a player.
An example string is shown here: `Id: 1 Points: 5 Active: True`

Table Q1(a)

The Game class has the following attributes: • _gamePoints is the amount of points to add or deduct from a player. • _winningPoints is the minimum points a player must have to win any game • _players is a dictionary of players playing the game, with player Id as key and player as value.
The Game class has the following methods: • __init__(self) initialises players as an empty dictionary. • Getter and setter methods for winning points. • deductPlayerPoints(self, playerId):bool locates a player with the parameter playerId. If the player is active, deduct points by the amount _gamePoints for that

player and return True. Return False if no deduction is made or if a player with a matching playerId cannot be located.

- getWinners(self):list returns a list of players with points at least _winningPoints. The method returns an empty list if there is no such player.
- addPlayer(self, player):bool adds the parameter player to the dictionary of players, if the dictionary does not contain the player already. If the add is successful, the method returns True. Otherwise, it returns False.
- removePlayer(self, playerId):Player removes a player with the parameter playerId from the dictionary, if the dictionary contain a player with the parameter playerId. If the remove is successful, the method returns the Player. Otherwise, it returns None.
- getPlayer(self, playerId):Player returns player with the parameter playerId from the dictionary, if the dictionary contain a player with the parameter playerId. Otherwise, it returns None.
- __str__(self) returns a string representation of a game, including the players, sorted by player id. An example string is shown here:

```
Game Player List:
Id: 1 Points: 5 Active: True
Id: 2 Points: 5 Active: True
Id: 3 Points: 5 Active: True
```

Table Q1(b)

(a) Refer to Table Q1(b). Give the rationale for the attributes and methods listed below being made class or instance members in the Game class.

- _winningPoints
- _players
- setWinningPoints
- deductPlayerPoints

(12 marks)

(b) Write a Python class definition for the Player class.

(7 marks)

(c) Construct and write a Python class definition for the Game class.

(13 marks)

(d) Write Python statements to perform the following actions:

(i) Create a Game object, game. Create a Player object, p and add p to game. Display the message:
- Successfully added if p was added to game, and
- Unable to add if p was not added.

(ii) Update winning points of a game to 12.

(iii) Deduct points from player, p and display the result of the deduction.

(3 marks)

Question 2

Study the class diagram in Figure Q2. The PetOwner class, the Pet class, the Dog class, and the Cat class are described in Table Q2(a) to Table Q2(e) respectively. Assume the PetOwner class has been implemented. You will only implement the Pet and Dog classes.

Demonstrate your understanding of how class hierarchy and association are used to organize information, and then construct the class hierarchy and association according to specification.

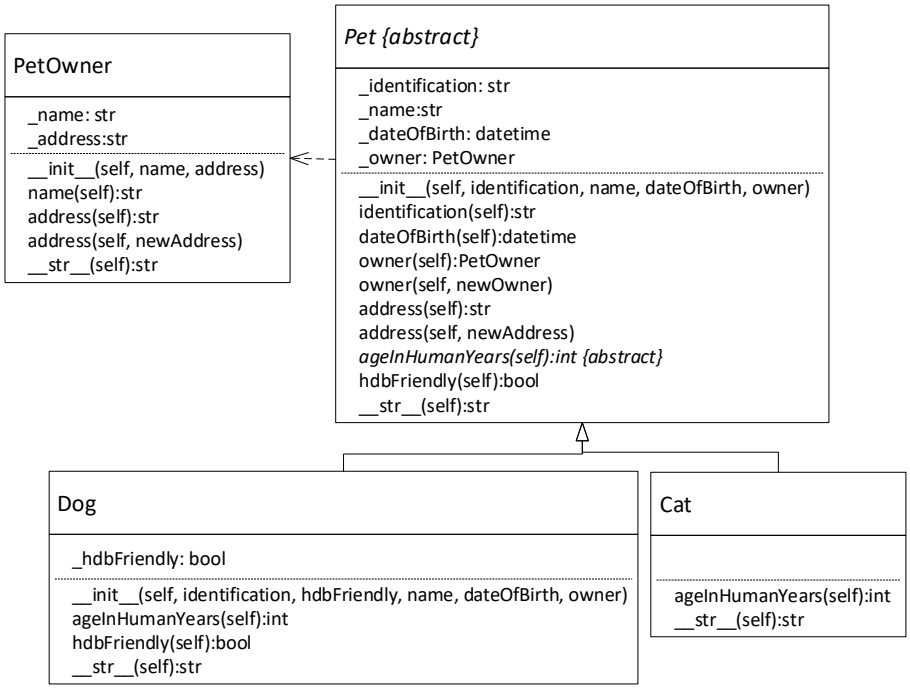

Figure Q2

The PetOwner class has the following attributes: • _name • _address
The PetOwner class has the following methods: • __init__(self, name, address) initialises a new pet owner with the parameters name and address. • Getter methods for name and address. • __str__(self) returns a string representation of a pet owner. An example string is shown here: `Name: Peter Address: 12 Dunbar Road`

Table Q2(a)

The Pet class is an abstract class. It has the following attributes: • _identification • _name • _dateOfBirth • _owner
The Pet class has the following methods: • __init__(self, identification, name, dateOfBirth, owner) initialises a new pet with the parameters identication, name, dateOfBirth and owner. The default value for the parameter owner is None.

- Getter methods for identification, date of birth, owner and address. The getter for address should return the owner's address.
- A setter method for owner and address. The setter method for address should update the owner's address.
- An abstract method ageInHumanYears(self) which returns the age of the pet in human years.
- A method hdbFriendly(self) which returns False.
- __str__(self) returns a string representation of a pet, including the age in human years and owner's detail if the pet has an owner, and the string None otherwise. Two example strings are shown here:

```
Id: C031 HDB Friendly: False
Name: Sparkle Date of Birth: 12 Jan 2018 Age in Human Year: 28
Owner Name: Peter Address: 12 Dunbar Road

Cat Id: C031 HDB Friendly: False
Name: Sparkle Date of Birth: 12 Jan 2018 Age in Human Year: 28
Owner None
```

Table Q2(b)

The Cat class is a subclass of the Pet class. It has the same attributes as Pet.

The Cat class has the following methods, in addition to the methods of the Pet class:

- ageInHumanYears(self) which returns the age of the cat in human years, computed according to these rules:
  - 0 to under first year since birth, 19 human years
  - between first to under second year since birth, 24 human years
  - add 4 human years for each subsequent year
- __str__(self) returns a string representation of a cat. An example string is shown here:

```
Cat Id: C031 HDB Friendly: False
Name: Sparkle Date of Birth: 12 Jan 2018 Age in Human Year: 28
Owner Name: Peter Address: 12 Dunbar Road
```

Table Q2(c)

The Dog class is a subclass of the Pet class. It has the following attributes in addition to the attributes of the Pet class:

- hdbFriendly records whether a dog is allowed in a HDB unit under HDB regulations.

The Dog class has the following methods, in addition to the methods of the Pet class:

- __init__(self, identification, hdbFriendly, name, dateOfBirth, owner) initialises a new pet with the parameters identification, hdbFriendly, name, dateOfBirth and owner.
- ageInHumanYears(self) which returns the age of the dog in human years, computed according to these rules:
  - 0 to under first year since birth, 15 human years
  - between first to under second year since birth, add 9 human years
  - add 5 human years for each subsequent year
- A method hdbFriendly(self) which returns whether a dog is hdbFriendly.
- __str__(self) returns a string representation of a dog. An example string is shown here:

```
Dog Id: D123 HDB Friendly: True
Name: Jackie Date of Birth: 07 Dec 2019 Age in Human Year: 15
Owner Name: Peter Address: 12 Dunbar Road
```

Table Q2(d)

(a) Differentiate between object composition and inheritance. Identify the relationship between the following pairs of classes and justify your choices.

(i) Pet and Owner

(ii) Pet and Cat

(3 marks)

(b) Define the Pet class.

(9 marks)

(c) Define the Dog class.

(8 marks)

Question 3

Apply the principles of object-oriented programming and develop an application where pets in a pet adoption agency are managed. Study the class diagram in Figure Q3 which is an extension of Figure Q2.

The PetAgencyException class and the PetAdoptionAgency are described in Table Q3(a) and Table Q3(b) respectively. Refer to Table Q2(a) to Table Q2(d) for the description of the other classes. You will implement the PetAgencyException and PetAdoptionAgency classes, and make modification to the getter and setter method for address in the Pet class.

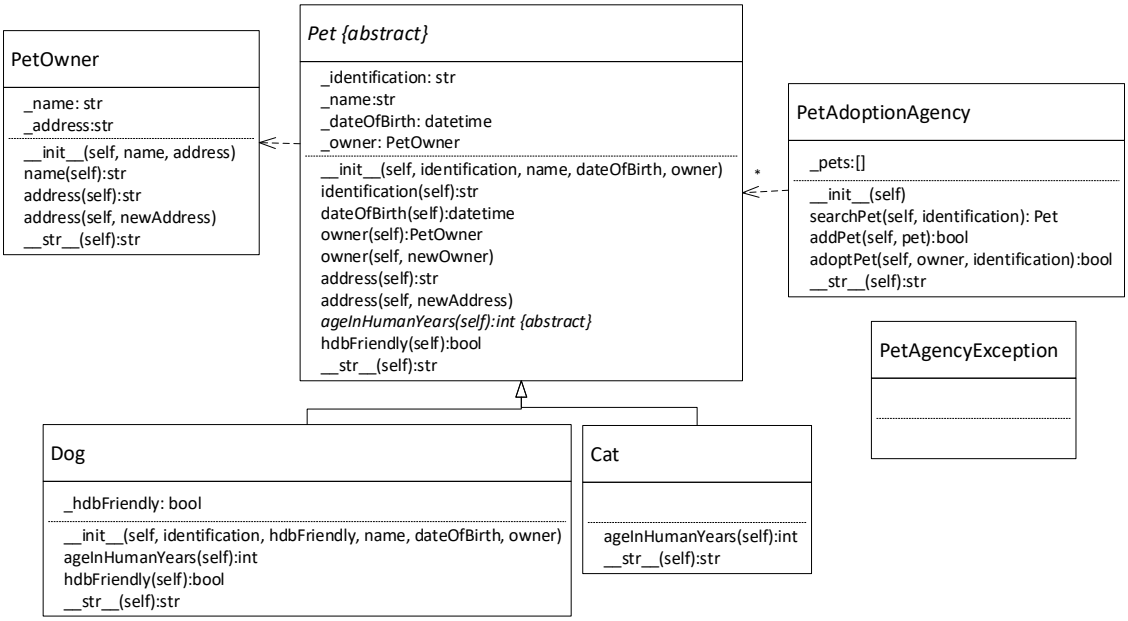

Figure Q3

The PetAgencyException class is a subclass of Exception. It has no additional attribute. A PetAgencyException object is raised when there is a business rule violation.
The PetAgencyException class has no additional method.

Table Q3(a)

The PetAdoptionAgency class has the following attributes: • _pets is a list of pets managed by the pet adoption agency.
The PetAdoptionAgency class has the following methods: • __init__(self) initialises a new pet adoption agency with an empty list of pets. • searchPet(self, identification):Pet returns a pet with the parameter identification if _pets contain a pet with the parameter identification. Otherwise, it returns None. • addPet(self, pet):bool adds the parameter pet to _pets, if _pets does not contain the pet already. If the add is successful, the method returns True. Otherwise, raises a PetAgencyException exception with the message Duplicate pet identification. Cannot add. • adopt(self, owner, identification):bool assigns the parameter owner as owner of the pet with the parameter identification and returns True. However, – if a pet cannot be located with the parameter identification, the method raises PetAgencyException exception with the message Incorrect pet identification. Cannot adopt. – if a pet already has an owner, the method raises PetAgencyException exception with the message Pet already has an owner. Cannot be adopted. – If the owner has already adopted 2 pets from the agency, disallow the adoption. The method raises PetAgencyException exception with the message Owner has adopted two pets already. Adoption quota reached. Note that owners are determined by their names and addresses. • updateAddress(self, identification, address):bool locates the pet with the parameter identification and updates its address with the parameter address. If a pet cannot be located with the parameter identification, the method raises PetAgencyException exception with the message Incorrect pet identification. Cannot update address.

Table Q3(b)

(a) Define the exception class PetAgencyException.

(1 mark)

(b) A pet may not have an owner. Modify getter and setter methods for address in the Pet class according to the description:

- Getter method: The method returns the owner address if the pet has an owner. Otherwise, the method raises a PetAgencyException exception with the message Pet has no owner. No owner address to retrieve.
- Setter method: The method updates the owner's address if the pet has an owner. Otherwise, the method raises a PetAgencyException exception with the message Pet has no owner. No owner address to update.

Submit only the getter and setter methods as answer. You do not need to submit the complete Pet class.

(4 marks)

(c) Implement the PetAdoptionAgency class.

(10 marks)

(d) Implement an application that starts by creating a PetAdoptionAgency with pets as shown in Table Q3(c), and then repeatedly allows the user to select one of the choices in the menu:

```
Menu
1. Update pet address
2. Adopt a pet
0. Exit
```

The choice is then processed as shown in the sample run (input underlined). Your application must handle the PetAgencyException as well as input errors.

Pet	Identification	HDB Friendly	Name	Date of Birth	Owner
Dog	D123	Yes	Jackie	7 Dec 2019	
Cat	C031	-	Sparkle	12 Jan 2018	
Cat	C017	-	Ginger	12 Jan 2018	Name: Peter Address: 12 Dunbar Road

Table Q3(c)

Sample run:

```
Menu
1. Update pet address
2. Adopt a pet
0. Exit
Enter a choice: a
Please enter a number for menu choice.
Menu
1. Update pet address
2. Adopt a pet
0. Exit
Enter a choice:
Please enter a number for menu choice.
Menu
1. Update pet address
2. Adopt a pet
0. Exit
Enter a choice: 1
Enter pet identification: D123
Enter new address: 2 Dunbar Road
Pet has no owner. No owner address to
update
Menu
1. Update pet address
2. Adopt a pet
0. Exit
Enter a choice: 1
Enter pet identification: D12
Enter new address: 2 Dunbar Road
Incorrect pet identification. Cannot
update address
Menu
1. Update pet address
2. Adopt a pet
0. Exit
Enter a choice: 1
Enter pet identification: C017
Enter new address: 2 Dunbar Road
Update address:  True
```

```
Menu
1. Update pet address
2. Adopt a pet
0. Exit
Enter a choice: 2
Enter owner name: Tom
Enter address: 34 Grey Lane
Enter pet identification: C017
Pet already has an owner. Cannot be
adopted
Menu
1. Update pet address
2. Adopt a pet
0. Exit
Enter a choice: 2
Enter owner name: Tom
Enter address: 34 Grey Lane
Enter pet identification: D12
Incorrect pet identification. Cannot
adopt
Menu
1. Update pet address
2. Adopt a pet
0. Exit
Enter a choice: 2
Enter owner name: Peter
Enter address: 2 Dunbar Road
Enter pet identification: C031
Adoption:  True
Menu
1. Update pet address
2. Adopt a pet
0. Exit
Enter a choice: 2
Enter owner name: Peter
Enter address: 12 Dunbar Road
Enter pet identification: D123
Owner has adopted two pets already.
Adoption quota reached
Menu
1. Update pet address
2. Adopt a pet
0. Exit
Enter a choice: 0
Application ends
```

(13 marks)

Question 4

Develop GUI for an application based on user requirements. The GUI converts the age of cats and dogs into human years based on the calculation rules in Table Q4(a).

Age in years	Cat	Dog
1	19	15
2	24	24
Per each year in excess of 2 years	+ 4	+5

Table Q4(a)

Figure Q4(a) shows the initial GUI.

When the user clicks on the button To Human Years, the GUI displays the human years for the selected animal type on the output label. Figure Q4(b) and figure Q4(c) show the GUI when the selected animal type is Cat and when the selected animal type is Dog, respectively.

When the user clicks on the button Clear, the GUI shows up as Figure Q4(a) again.

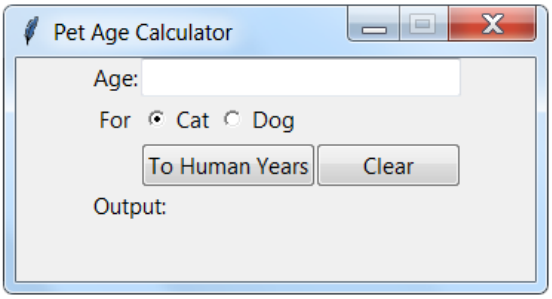

Figure Q4(a)

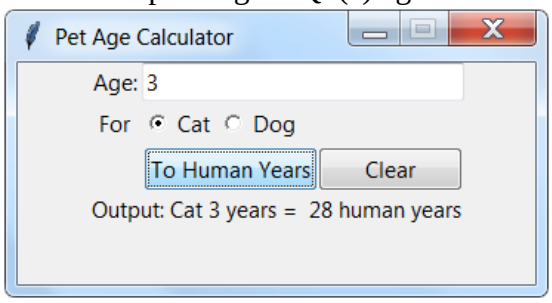

Figure Q4(b)

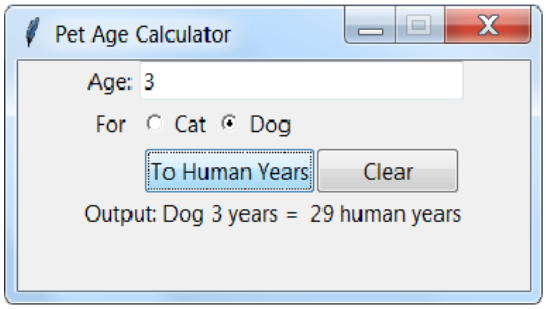

Figure Q4(c)

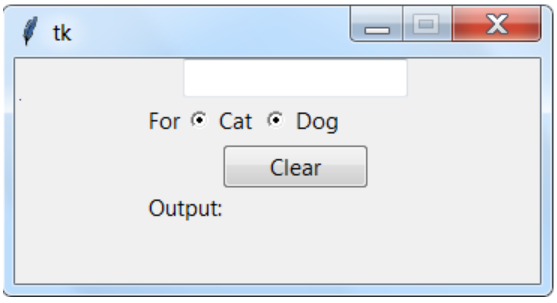

Figure Q4(d)

You are given the partial code. Note that the code does not produce the GUI in Figure Q4(a). Instead, it produces a GUI in Figure Q4(d):

```
import tkinter as tk
from tkinter import ttk
class PetAgeCalculatorGui:
    def __init__(self):
        self._win = tk.Tk()
        self._win.resizable(False, False)
        self.create_widgets()
        self._win.geometry('350x150')
        self._win.mainloop()
    def create_widgets(self):
        dataFrame = ttk.Frame(self._win)
        self._age_lbl = ttk.Label(dataFrame, text="Age:")
        self._ageValue_Ety = ttk.Entry(dataFrame, width=18)
        self._ageValue_Ety.grid(column=1, row=0, sticky=tk.EW)
        for_lbl = ttk.Label(dataFrame, text="For")
        for_lbl.grid(column=0, row=1)
        radioFrame = ttk.Frame(dataFrame)
        radioFrame.grid(column=1, row=1, sticky=tk.EW)
        cat_rdbtn = tk.Radiobutton(radioFrame, text = 'Cat')
        dog_rdbtn = tk.Radiobutton(radioFrame, text = 'Dog')
        cat_rdbtn.grid(column=1, row=0, sticky=tk.W)
        dog_rdbtn.grid(column=2, row=0, sticky=tk.W)
        actionFrame = ttk.Frame(dataFrame)
        actionFrame.grid(column=1, row=3)
        self._convert_btn = ttk.Button(actionFrame, text="To Human Years")
        self._clear_btn = ttk.Button(actionFrame, text="Clear")
        self._clear_btn.pack(side = tk.LEFT)
        self._output_lbl = ttk.Label(dataFrame, text="Output:",
justify=tk.LEFT)
        self._output_lbl.grid(column=0, row=4, columnspan = 2, sticky=tk.EW)
        dataFrame.pack(side = tk.TOP)
```

(a) Modify and/or add code to the partial code to make the GUI appear as shown in Figure Q4(a).

(7 marks)

(b) Implement event handling so that the GUI can respond to left button clicks on both buttons.

(10 marks)

----- END OF PAPER -----